



Angular Training Material

A Complete Guide — From Basics to Full Project

TypeScript | Components | Routing | Services | HTTP | CRUD

Contents

- 1. Angular Intro**
- 2. How Angular Works**
- 3. Installation and Project Setup**
- 4. Your First Angular App**
- 5. Understanding Components**
 - 5.1 Component Basics
 - 5.2 Component Communication (@Input / @Output)
 - 5.3 Lifecycle Hooks
- 6. Data Binding & Directives**
 - 6.1 Types of Data Binding
 - 6.2 Built-in Directives
- 7. Angular Routing**
 - 7.1 Setting Up Routes
 - 7.2 Route Parameters & Guards
- 8. Services & Dependency Injection**
- 9. HTTP Client & API Integration**
- 10. Angular Forms**
 - 10.1 Template-Driven Forms
 - 10.2 Reactive Forms
- 11. Folder Structure in Angular**
 - 11.1 Default Structure
 - 11.2 Feature-Based Structure (Industry Standard)
- 12. Commonly Used Third-Party Libraries**
- 13. CRUD with Login Project**
- 14. Environment Variables in Angular**

1. Angular Intro

Angular is a **TypeScript-based open-source front-end web framework** developed and maintained by Google. It is used to build dynamic, single-page web applications (SPAs) with a structured and scalable architecture. Angular provides everything out-of-the-box: routing, forms, HTTP client, animations, testing utilities, and more.

- **Without Angular:** You write plain HTML + JS, managing the DOM manually.
- **With Angular:** You build component-based UIs that are reactive and maintainable.
- Angular uses **TypeScript**, giving you type safety and modern JavaScript features.
- It follows the **MVC / MVVM** architecture pattern.

Key Concepts at a Glance:

Concept	Description
Component	The building block of every Angular UI
Module	Groups related components/services together
Service	Reusable business logic, shared across components
Directive	Extends HTML with custom behaviour
Pipe	Transforms displayed data (e.g., date, currency)
Router	Navigates between views without page reload

2. How Angular Works

Angular applications run in the browser. When a user opens an Angular app, the browser loads a single HTML file (index.html) and Angular takes over the rendering and navigation without full page reloads.

Step	Phase	Description
Step 1	Bootstrap	Angular starts from main.ts, which bootstraps AppModule.
Step 2	Module Load	AppModule declares all components, imports, and providers.
Step 3	Component Tree	Angular builds a tree of components starting from AppComponent.
Step 4	Template Rendering	Angular compiles templates and binds data to the DOM.
Step 5	Change Detection	Angular watches for data changes and updates the view automatically.
Step 6	Routing	The Router intercepts URL changes and renders the matching component.
Step 7	HTTP / APIs	Services use HttpClient to fetch data; components subscribe and display it.

3. Installation and Project Setup

To start building Angular applications, you need **Node.js** and **npm** installed first. Then install the Angular CLI (Command Line Interface) globally.

Install Angular CLI:

```
npm install -g @angular/cli

# Verify installation

ng version
```

Create a New Angular Project:

```
ng new my-angular-app

# CLI will ask:

# ? Would you like to add Angular routing? (y/N) --> y

# ? Which stylesheet format? --> CSS / SCSS
```

Run the Development Server:

```
cd my-angular-app

ng serve

# Open browser: http://localhost:4200
```

The live-reload server automatically refreshes the browser whenever you save a file.

4. Your First Angular App

After scaffolding, open **src/app/app.component.ts**. Every Angular component has three key parts: a TypeScript class, an HTML template, and CSS styles.

app.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'Hello, Angular!';
}
```

app.component.html

```
<h1>{{ title }}</h1>

<p>Welcome to my first Angular app!</p>
```

The **{{ }}** double curly braces are Angular's interpolation syntax — they display the value of the component's property in the template.

5. Understanding Components

A **component** is the fundamental building block of an Angular application. Everything visible on screen is a component. Angular apps are essentially a tree of components. Each component controls a patch of the screen called a view.

5.1 Component Basics

Generate a new component using CLI:

```
ng generate component user

# short form:

ng g c user
```

This creates 4 files:

- user.component.ts
- user.component.html
- user.component.css
- user.component.spec.ts

5.2 Component Communication (@Input / @Output)

@Input passes data from parent to child component. **@Output** emits events from child to parent.

```
// child.component.ts

import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({ selector: 'app-child', template: `

  <p>{{ message }}</p>

  <button (click)='sendEvent()'>Click Me</button>` })

export class ChildComponent {

  @Input() message: string = '';

  @Output() clicked = new EventEmitter<string>();

  sendEvent() {

    this.clicked.emit('Hello from child!');
```

```
}  
  
}  
  
<!-- parent.component.html -->  
  
<app-child [message]='parentMsg'  
          (clicked)='onChildClick($event)'  
</app-child>
```

5.3 Lifecycle Hooks

Angular calls lifecycle hook methods at specific moments in a component's life.

Hook	When it runs
<code>ngOnInit()</code>	After component is initialized — most common hook
<code>ngOnChanges()</code>	When <code>@Input</code> properties change
<code>ngDoCheck()</code>	On every change detection cycle
<code>ngAfterViewInit()</code>	After component's view is fully initialized
<code>ngOnDestroy()</code>	Just before component is destroyed (cleanup)

6. Data Binding & Directives

6.1 Types of Data Binding

Type	Syntax	Direction	Example
Interpolation	{{ value }}	TS → Template	{{ user.name }}
Property Binding	[property]='value'	TS → Template	[src]='imageUrl'
Event Binding	(event)='handler()'	Template → TS	(click)='save()'
Two-Way Binding	[(ngModel)]='prop'	Both directions	[(ngModel)]='email'

6.2 Built-in Directives

Angular provides structural and attribute directives to manipulate the DOM.

```
<!-- *ngIf – Conditionally renders element -->

<p *ngIf='isLoggedIn'>Welcome back!</p>

<!-- *ngFor – Loops over a list -->

<li *ngFor='let user of users'>{{ user.name }}</li>

<!-- *ngSwitch – Switch-case rendering -->

<div [ngSwitch]='role'>

  <p *ngSwitchCase="'admin'">Admin Panel</p>

  <p *ngSwitchDefault>User View</p>

</div>

<!-- [ngClass] – Dynamic CSS classes -->

<div [ngClass]='{"active": isActive, "error": hasError}'></div>

<!-- [ngStyle] – Dynamic inline styles -->

<p [ngStyle]='{"color": textColor, "font-size": "14px"}'></p>
```

7. Angular Routing

Angular's Router enables navigation between views in a Single Page Application (SPA) without reloading the entire page. Routes map URL paths to components.

7.1 Setting Up Routes

```
// app-routing.module.ts

import { NgModule } from '@angular/core';

import { RouterModule, Routes } from '@angular/router';

import { HomeComponent } from '../home/home.component';

import { UserComponent } from '../user/user.component';

import { LoginComponent } from '../login/login.component';

const routes: Routes = [

  { path: '',      component: HomeComponent },

  { path: 'users', component: UserComponent },

  { path: 'login', component: LoginComponent },

  { path: '**',    redirectTo: '' } // wildcard

];

@NgModule({

  imports: [RouterModule.forRoot(routes)],

  exports: [RouterModule]

})

export class AppRoutingModule {}
```

```
<!-- app.component.html -->

<nav>

  <a routerLink="/">Home</a>

  <a routerLink="/users">Users</a>
```

```
</nav>

<router-outlet></router-outlet> <!-- renders matched component -->
```

7.2 Route Parameters & Guards

```
// Route with parameter

{ path: 'users/:id', component: UserDetailsComponent }

// Read param in component

import { ActivatedRoute } from '@angular/router';

constructor(private route: ActivatedRoute) {}

ngOnInit() {

  const id = this.route.snapshot.paramMap.get('id');

}

// Auth Guard – protects routes

// ng generate guard auth

canActivate(): boolean {

  return !!localStorage.getItem('token');

}
```

8. Services & Dependency Injection

A **service** is a class that holds reusable business logic, data access, or shared state. It is separate from components to keep components lean. **Dependency Injection (DI)** is Angular's way of providing a service instance to a component automatically.

```
// Generate service

ng generate service user

// user.service.ts

import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // Singleton – available app-wide
})

export class UserService {

  private users = ['Alice', 'Bob', 'Carol'];

  getUsers(): string[] {

    return this.users;

  }

}
```

```
// user.component.ts – inject the service

constructor(private userService: UserService) {}

ngOnInit() {

  this.users = this.userService.getUsers();

}
```

9. HTTP Client & API Integration

Angular's **HttpClientModule** provides a powerful way to communicate with backend APIs. It returns Observables (from RxJS) instead of Promises.

Step 1: Import HttpClientModule in app.module.ts

```
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [ HttpClientModule ]
})
```

Step 2: Use HttpClient in a Service

```
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class ApiService {
  private baseUrl = 'http://localhost:3000/api';

  constructor(private http: HttpClient) {}

  getUsers(): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseUrl}/users`);
  }

  createUser(data: any): Observable<any> {
    return this.http.post(`${this.baseUrl}/users`, data);
  }

  updateUser(id: string, data: any): Observable<any> {
    return this.http.put(`${this.baseUrl}/users/${id}`, data);
  }
}
```

```
    }

    deleteUser(id: string): Observable<any> {

        return this.http.delete(`${this.baseUrl}/users/${id}`);

    }

}
```

Step 3: Subscribe in Component

```
this.apiService.getUsers().subscribe({

    next: (data) => { this.users = data; },

    error: (err) => { console.error(err); }

});
```

10. Angular Forms

Angular provides two approaches to handling user input through forms.

10.1 Template-Driven Forms

Simple and quick. Logic lives in the template using directives. Best for simple forms.

```
<!-- Import FormsModule in app.module.ts -->

<form (ngSubmit)='onSubmit()' #myForm='ngForm'>

  <input name='email' [(ngModel)]='email' required />

  <input name='password' [(ngModel)]='password' type='password' />

  <button type='submit' [disabled]='!myForm.valid'>Login</button>

</form>
```

10.2 Reactive Forms

More powerful and explicit. Logic is defined in the TypeScript class. Best for complex, dynamic forms.

```
// Import ReactiveFormsModule in app.module.ts

import { FormBuilder, FormGroup, Validators } from '@angular/forms';

export class LoginComponent {

  loginForm: FormGroup;

  constructor(private fb: FormBuilder) {

    this.loginForm = this.fb.group({

      email:    ['', [Validators.required, Validators.email]],

      password: ['', [Validators.required, Validators.minLength(6)]]

    });

  }

  onSubmit() {
```

```
    if (this.loginForm.valid) {  
        console.log(this.loginForm.value);  
    }  
}  
}
```

```
<!-- Reactive form template -->  
  
<form [formGroup]='loginForm' (ngSubmit)='onSubmit()'  
    <input formControlName='email' placeholder='Email' />  
    <span *ngIf="loginForm.get('email')?.invalid">Invalid email</span>  
    <input formControlName='password' type='password' />  
    <button [disabled]='loginForm.invalid'>Login</button>  
</form>
```


11. Folder Structure in Angular

11.1 Default CLI Structure

```
my-angular-app/  
  
■■■ src/  
  
■   ■■■ app/  
  
■   ■   ■■■ app.component.ts  
■   ■   ■■■ app.component.html  
■   ■   ■■■ app.component.css  
■   ■   ■■■ app.module.ts  
■   ■   ■■■ app-routing.module.ts  
  
■   ■■■ assets/  
■   ■■■ environments/  
■   ■   ■■■ environment.ts  
■   ■   ■■■ environment.prod.ts  
  
■   ■■■ index.html  
■   ■■■ main.ts  
  
■■■ angular.json  
■■■ tsconfig.json  
■■■ package.json
```

11.2 Feature-Based Structure (Industry Standard)

```
src/app/  
  
■■■ core/                                # Singleton services, guards, interceptors  
  
■   ■■■ services/  
  
■   ■   ■■■ auth.service.ts  
■   ■   ■■■ api.service.ts
```

```

■   ■■■ guards/

■   ■   ■■■ auth.guard.ts

■   ■■■ interceptors/

■       ■■■ token.interceptor.ts

■

■■■ shared/                                # Shared components, pipes, directives

■   ■■■ components/

■   ■   ■■■ loader/

■   ■■■ pipes/

■       ■■■ date-format.pipe.ts

■

■■■ features/                              # Feature modules

■   ■■■ auth/

■   ■   ■■■ login/

■   ■   ■■■ register/

■   ■   ■■■ auth.module.ts

■   ■■■ user/

■       ■■■ user-list/

■       ■■■ user-detail/

■       ■■■ user.module.ts

■

■■■ app.module.ts

■■■ app-routing.module.ts

```

Folder	Purpose
core/	Singleton services loaded once (auth, API, interceptors)
shared/	Reusable UI components, pipes, and directives
features/	Feature-specific modules — each self-contained

<code>assets/</code>	Static files: images, fonts, icons
<code>environments/</code>	Build-time config (API URLs per environment)

12. Commonly Used Third-Party Libraries

Library	Install / Status	Use Case
Angular Material	<code>npm install @angular/material</code>	Official UI component library (buttons, tables, dialogs, etc.)
NgRx	<code>npm install @ngrx/store @ngrx/effects</code>	State management using Redux pattern for complex apps
RxJS	Built-in with Angular	Reactive programming library — Observables, operators
Axios / HttpClient	Built-in HttpClient (preferred)	HTTP requests; Angular's HttpClient is recommended
NGX-Toastr	<code>npm install ngx-toastr</code>	Toast notifications (success, error, warning, info)
JWT Decode	<code>npm install jwt-decode</code>	Decode JWT tokens in the frontend
Lodash	<code>npm install lodash</code>	Utility functions for arrays, objects, strings
NGX-Translate	<code>npm install @ngx-translate/core</code>	Internationalization (i18n) / multi-language support
Chart.js / ng2-charts	<code>npm install ng2-charts chart.js</code>	Data visualization — bar, pie, line charts
Angular Fire	<code>npm install @angular/fire</code>	Firebase integration — auth, Firestore, hosting

13. CRUD with Login Project

We will build a complete Angular frontend that connects to a Node.js/Express backend. This project covers Register, Login (JWT), and full CRUD for users.

Install Dependencies:

```
ng new crud-app --routing

cd crud-app

npm install jwt-decode
```

Auth Service — auth.service.ts

```
import { Injectable } from '@angular/core';

import { HttpClient } from '@angular/common/http';

import { Router } from '@angular/router';

@Injectable({ providedIn: 'root' })

export class AuthService {

  private api = 'http://localhost:3000/api';

  constructor(private http: HttpClient, private router: Router) {}

  register(data: any) {

    return this.http.post(`${this.api}/register`, data);

  }

  login(data: any) {

    return this.http.post<{token: string}>(`${this.api}/login`, data);

  }

  saveToken(token: string) {

    localStorage.setItem('token', token);

  }

}
```

```
}

isLoggedIn(): boolean {

    return !!localStorage.getItem('token');

}

logout() {

    localStorage.removeItem('token');

    this.router.navigate(['/login']);

}

}
```

Login Component — login.component.ts

```
export class LoginComponent {

    email = ''; password = '';

    constructor(private auth: AuthService,

                private router: Router) {}

    onLogin() {

        this.auth.login({ email: this.email, password: this.password })

            .subscribe({

                next: (res) => {

                    this.auth.saveToken(res.token);

                    this.router.navigate(['/users']);

                },

                error: () => alert('Login failed')

            });

    }

}
```

```
}
```

User Service — user.service.ts

```
getUsers()          { return this.http.get(`${this.api}/users`,
                                { headers: this.getHeaders() }); }

createUser(data)     { return this.http.post(`${this.api}/users`, data,
                                { headers: this.getHeaders() }); }

updateUser(id, data){ return this.http.put(`${this.api}/users/${id}`, data,
                                { headers: this.getHeaders() }); }

deleteUser(id)       { return this.http.delete(`${this.api}/users/${id}`,
                                { headers: this.getHeaders() }); }

private getHeaders() {
    return { Authorization: `Bearer ${localStorage.getItem('token')}` };
}
```

HTTP Interceptor — token.interceptor.ts

An interceptor automatically attaches the JWT token to every outgoing request.

```
intercept(req: HttpRequest<any>, next: HttpHandler) {

    const token = localStorage.getItem('token');

    if (token) {

        req = req.clone({

            setHeaders: { Authorization: `Bearer ${token}` }

        });

    }

    return next.handle(req);

}
```

API Endpoints Reference:

Method	Endpoint	Description	Auth Required
POST	/api/register	Register new user	No
POST	/api/login	Login — get JWT token	No
GET	/api/users	Get all users	Yes
PUT	/api/users/:id	Update a user	Yes
DELETE	/api/users/:id	Delete a user	Yes

14. Environment Variables in Angular

Angular provides built-in environment files to manage configuration values that differ between development and production (e.g., API base URLs, feature flags).

Default Environment Files:

```
src/environments/  
  
■■■ environment.ts          # Development config  
  
■■■ environment.prod.ts     # Production config
```

environment.ts (Development)

```
export const environment = {  
  
  production: false,  
  
  apiUrl: 'http://localhost:3000/api',  
  
  appName: 'MyApp Dev'  
  
};
```

environment.prod.ts (Production)

```
export const environment = {  
  
  production: true,  
  
  apiUrl: 'https://api.myapp.com/api',  
  
  appName: 'MyApp'  
  
};
```

Using Environment Variables in a Service:

```
import { environment } from '../../../environments/environment';  
  
@Injectable({ providedIn: 'root' })  
  
export class ApiService {  
  
  private baseUrl = environment.apiUrl; // auto-swapped on build
```

```
getUsers() {  
  
    return this.http.get(`${this.baseUrl}/users`);  
  
}  
  
}
```

Building for Production:

```
ng build                # Development build  
  
ng build --configuration production # Production build  
  
                        # Angular swaps environment.ts  
  
                        # with environment.prod.ts automatically
```

Tip	Details
Never hardcode URLs	Always use environment.apiUrl in services
No sensitive secrets	Angular env files are bundled into JS — visible to users
Use .env for build	CI/CD pipelines can inject values at build time
fileReplacements	angular.json controls which env file is swapped

Summary: Angular is a complete, opinionated framework by Google. You now have a solid foundation covering components, routing, services, HTTP, forms, folder structure, third-party libraries, a full CRUD project, and environment configuration. Keep building!